RECETA MAESTRA v1 (Standard-Grade) — uAgri ERC-7943 Agricultural Exploitation & Crowdfarming Kit

Objetivo: un kit **open-source**, **multipaís**, con **conformance suite** y perfiles de despliegue "copy-paste", que estandariza **financiación + rendimiento + trazabilidad + (opcional) comercio/entrega física**, usando **ERC-7943** como capa de enforcement para participaciones fungibles por campaña.

> **Compilación de referencia:** Solidity **0.8.33** (pinned).
> **Principio rector:** el activo es *off-chain*, el sistema es **operado y auditable (trust-minimized, no trustless)**: roles separados, proofs/attestations, dual-control, timelock, historial inmutable y tests/invariants.

---

0) Base normativa innegociable (ERC-7943 fungible)

0.1 MUST del token ERC-7943

- Implementar **ERC-20** + **IERC7943Fungible** + **ERC-165**.
- `canTransact(account)`, `canTransfer(from,to,amount)`, `getFrozenTokens(account)`:
  - **MUST NOT revert**
  - **MUST NOT touch storage** (no writes; idealmente no reads del propio token más allá de balance/frozen en memoria/cached).
  - Si un módulo externo falla: **safe-staticcall + fail-closed** (devolver `false` o un valor seguro) sin revertir.
- `canTransfer` MUST evaluar: `amount <= balance(from) - frozen(from)` **y** `canTransact(from)` **y** `canTransact(to)`.
- `setFrozenTokens(account, amount)` y `forcedTransfer(from,to,amount)`:
  - **MUST** ser administrativas (RBAC)
  - **MUST** emitir eventos estándar
  - **MUST** revertir con errores claros en inválidos.
- `forcedTransfer` (single-party / judicial fuerte):
  - **MAY** bypass `canTransfer`,

    - pero si toca tokens congelados: **unfreeze-before-transfer** + evento `Frozen` previo y luego `ForcedTransfer`.
- Enforcement: `transfer/transferFrom` (y mint/burn donde aplique) **MUST** respetar `canTransfer/canTransact` + pausas.

0.2 Recomendación de "no revert en views"

- Todas las views ERC-7943 **fail-closed** si el compliance/oracle revierte, no responde, o consume demasiado gas.

---

1) Qué hace a uAgri "estándar de facto"

Un estándar "de facto" no es solo contratos. Este kit incluye:

1. **Interfaces internas versionadas** (módulos pluggable).
2. **Perfiles de despliegue** predefinidos (sin editar código).
3. **Conformance suite**: ERC-7943 + invariantes agrícolas + multichain si aplica.
4. **Threat model + runbooks** (roles, llaves, emergencias, upgrades, desastre).
5. **Documentación estándar** tipo RFC (`MUST/SHOULD/MAY`).
6. **Extensiones oficiales** (marketplace, delivery, bridges, 4626 wrapper opcional, etc.).

---

2) Decisiones fijadas (resultado de tus respuestas)

2.1 Unidad económica y significado del token

- **Unidad económica estándar: 1 campaña/temporada/lote económico = 1 token ERC-7943** ("AgriShare").
- **El ERC-7943 representa: participación económica pro-rata** en los proceeds netos de esa campaña.
- **Entrega física y comercio: extensiones** (no core) mediante receipts (ERC-1155/721) + marketplace adaptado.

2.2 Funding/Redemption y payout

- **Core MUST:** funding + redemption (request/process).
- **Instant SHOULD:** perfil retail opcional.
- **SettlementAsset por campaña: SHOULD** existir un activo canónico (típicamente stable).

- Multi-asset deposits **MAY** mediante `PaymentAdapter` (swap/quote).
- **Payout MUST:** en settlementAsset (y incentive token **MAY** separado).

2.3 Mercado secundario y transfer policy

- Mercado secundario libre (no whitelist global).
- Lock-ups/ventanas **MUST** estar soportados por compliance, pero **default SHOULD** libre.

2.4 Disaster y compliance

- Desastre: catálogo base de hazards + extensible por jurisdicción.
- Activación: **rápida con TTL** por `DISASTER_ADMIN` + **confirmación fuerte** (multisig/timelock) para extender o ejecutar hard actions.
- **MUST:** freeze por cuenta + **restricted mode por campaña**.
- **Compensación MAY:** `InsurancePool` (SHOULD como extensión oficial).
- KYC/AML: **híbrido** (gating fuerte en mint/redeem/claim/tier), compliance pluggable.
- EUDR: **EU Profile** (módulo), no core global.

2.5 Trazabilidad y oráculos

- Trazabilidad “máxima” sin matar gas:
  - hashes+eventos MUST; merkle roots SHOULD; ZK/NFT receipts MAY.
- Oráculos MUST: harvest, sales/proceeds evidence, custody/inventory, disaster evidence.
- **EIP-712 base MUST**, Chainlink/adapters MAY.

2.6 Multichain

- Mainnet + Base/Arbitrum/Polygon.
- Bridge con invariante de supply: **MUST como módulo activable** (perfil multichain).

---

3) Perfiles de despliegue (copy-paste)

Perfil A — Immutable / No-upgrade (máxima confianza)

- Campaña (token + módulos) con direcciones inmutables en constructor.
- Cambios de política/módulo → nueva campaña / nueva versión de template.

Perfil B — Upgradeable (UUPS “institucional”)

- Token proxy UUPS (por campaña o por template).

- **Guardian multisig** + **Timelock**:
    - emergencias: multisig (sin delay)
    - políticas/upgrades: timelock (delay obligatorio)

Perfil C — Hybrid (token fijo, módulos reemplazables)

- Token no upgradeable.
- `Compliance/Oracle/Disaster/Bridge` reemplazables vía timelock + guardian.

Perfil D — Factory + Clones (DEFAULT)

- **Factory** despliega campañas como **clones EIP-1167**:
    - `AgriShareToken` (ERC-7943) clone
    - `CampaignTreasury` clone
    - config fija + pointers a módulos/templates compartidos
- Ideal para volumen M/L y despliegue L2-first.

---

4) Arquitectura multicontrato (cerrada)

4.1 CORE MUST (por campaña)

1. **AgriShareToken.sol**
    - ERC20 + ERC-7943 + ERC165
    - Enforcement en transfer/transferFrom (+ mint/burn si aplica)
    - Hooks `_beforeTokenTransfer/_afterTokenTransfer`
    - Views ERC-7943 **no revert** + **fail-closed** via safe-staticcall a:
        - `ComplianceModule`
        - `Pause/DisasterModule` (estado campaña)
    - Forced transfer “judicial fuerte” conforme a reglas fijadas.
2. **FreezeManager.sol**
    - `frozen[account]` y `setFrozenTokens(account, amount)` (rol dedicado)
    - Política: freeze afecta transfer; claim/redeem según flags campaña.
3. **ForcedTransferController.sol**
    - `forcedTransfer(from,to,amount)`
    - Policy fija: judicial fuerte (bypass) + unfreeze-before-transfer si necesario.
4. **RoleManager.sol**
   Roles mínimos recomendados (separación de deberes):
    - `DEFAULT_ADMIN` (ideal: timelock)
    - `FARM_OPERATOR`
    - `COMPLIANCE_OFFICER`

    - `DISASTER_ADMIN`
    - `ORACLE_UPDATER`
    - `CUSTODY_ATTESTER`
    - `REGULATOR_ENFORCER` (forcedTransfer/freeze extremo)
    - `TREASURY_ADMIN`
    - `GUARDIAN` (pause)
    - `UPGRADER` (si aplica)
    - `BRIDGE_OPERATOR` (si aplica)
    - Two-step para admin y roles críticos.
5. **EmergencyPause.sol**
   Pausas granulares:
    - `pauseTransfers`
    - `pauseFunding`
    - `pauseRedemptions`
    - `pauseClaims`
    - `pauseOracleUpdates`
      En pausa de transfers: `canTransfer/canTransact` → false (sin revert).

4.2 CAMPAIGN OPS MUST

6. **AgriCampaignRegistry.sol**
    - `campaignId`, `plotRef`, `subPlotId/areaBps (optional)`, `cropType`, `start/end`, `settlementAsset`, `fundingCap`, `docsRootHash`, `jurisdictionProfile`, `state`
    - State machine cerrada: FUNDING → ACTIVE → HARVESTED → SETTLED → CLOSED
7. **FundingManager.sol**
    - Request/Process MUST, Instant SHOULD (config)
    - soporta multi-asset deposits via adapter (MAY)
    - Fees simples (subscription/redemption) MUST
    - CEI + pull-payments cuando aplique
8. **SettlementQueue.sol** (Request/Process)
    - Requests: `id, account, kind(deposit/redeem), amountIn/amountShares, constraints, deadline, status`
    - `requestDeposit`, `requestRedeem`, `cancel`, `batchProcess`
    - Anti-spam: max pending per account, expiries, no double process
    - Gas: packing en slots (fase posterior, tras tests verdes)
9. **Treasury/CampaignTreasury.sol**
    - recibe settlementAsset

    - controla pagos de proceeds/insurance
    - limita acciones según `restricted mode` y proofs

10. **Distribution MUST (elige 1; ambos incluidos)**

- **YieldAccumulator.sol (default)**: `accRewardPerShare + rewardDebt`, `notifyReward(amount)`, `claim()`
- **SnapshotModule.sol (exacto)**: snapshots por epoch, claims por epoch

Norma: proceeds-based (cashflow real). Los oráculos aportan evidencia, pero el `notifyReward` debe estar respaldado por fondos efectivamente recibidos (o escrow verificable según perfil).

4.3 DISASTER MUST (agrícola)

11. **DisasterModule.sol**

- `declareDisaster(hazardCode, severity, reasonHash, ttl)` por `DISASTER_ADMIN`
- `confirm/extend/escalate` solo por multisig/timelock (dual-control) para hard actions
- Niveles:
    - Alert (suave)
    - Restricted (pausa redemption/claim por defecto)
    - HardFreeze (pausa transfers; solo acciones judiciales/reestructuración)
- TTL obligatorio para activación rápida no confirmada.

12. **InsurancePool.sol (SHOULD extension, core-compatible)**

- compensación pro-rata por snapshot/epoch
- pull-claims
- funding del pool por reserves, premiums, aportes externos
- ejecutable solo en evento confirmado (dual-control).

4.4 COMPLIANCE MUST (pluggable multipaís)

13. **ComplianceModuleV1.sol** (Policy Engine)
    Capabilities mínimas (MUST):

- sanctions deny
- jurisdiction allow/deny
- investor tier + caps
- holding cap
- lockupUntil (por cuenta o por mint lot)
- transfer windows (campaña, opcional)

- daily transfer caps (opcional pero recomendado)
- corridors (from→to) SHOULD
- venues allowlist MAY

14. **IdentityAttestation.sol** (onboarding)

- EIP-712 `register(account, payload, deadline, sig)`
- payload: `jurisdiction, tier, flags, expiry, providerId, lockupUntil`
- nonces anti-replay por `(account, providerId)`
- expiración/renovación

15. **Regional Profiles (EU / etc.)**

- **EUProfileEUDRCompliance.sol** (SHOULD)
- Otros perfiles como módulos futuros.

4.5 TRACEABILITY MUST (máximo nivel sin gas infierno)

16. **TraceabilityRegistry.sol**

- `TraceEvent`: campaignId/plotRef/lotId, eventType, dataHash, issuer, timeRange, pointer
- Guarda mínimo, emite eventos; evita storage grande.

17. **BatchMerkleAnchor.sol (SHOULD)**

- `anchorRoot(campaignId, batchType, root, fromTs, toTs, issuer)`
- Permite proofs off-chain por leaf.

18. **DocumentRegistry.sol (SHOULD fuerte)**

- `registerDoc(docType, docHash, issuer, issuedAt, refs...)`
- para auditorías, certificados, EUDR packs, custodia, ventas, pólizas.

4.6 ORACLES MUST (EIP-712 base)

19. **HarvestOracle.sol**

- attestations: quantity, quality, lotIds, epoch, asOf, validUntil, reportHash

20. **Sales/ProceedsOracle.sol**

- evidencia de ventas/settlement: amounts, lots, epoch, reportHash

21. **CustodyOracle.sol**

- inventario por lot/warehouse, asOf, validUntil, reportHash
- gating: sin custodia vigente → no process redeem/claim (si perfil lo activa)

22. **DisasterEvidenceOracle.sol (MAY/SHOULD)**

- meteorológico/satélite/aseguradora: severity, region, reportHash

23. **OracleAdapter.sol (MAY)**

- Chainlink/feeds → normalización a interfaz interna / attestation “as-if”.

Garantías MUST para oráculos:

- anti-replay, epoch monotónico, freshness window, reportHash obligatorio, historial inmutable (no sobrescribir sin rastro).

4.7 MARKETPLACE & DELIVERY (EXTENSIONS oficiales)

24. **ReferenceMarketplace.sol (EXTENSION)**
- fixed-price listings + partial fills (MUST)
- bids simples (SHOULD)
- subastas/RFQ (MAY)
- pagos en ERC20 stables (base), multi-asset opcional
- respeta enforcement ERC-7943 (`canTransfer/canTransact`) siempre

25. **PhysicalDeliveryModule.sol (EXTENSION)**
- burn shares → emite **DeliveryReceipt** (ERC-1155 recomendado)
- receipt canjeable off-chain con custodia/warehouse (hashes + attestations)
- compatible con compliance (KYC requerido para redeem físico típicamente)

4.8 MULTICHAIN (MUST como módulo activable por perfil)

26. **BridgeAdapterCanonical.sol**
- “canonical chain per campaign”
- soporta lock/mint o burn/mint
- invariante de supply cross-chain
- replay protection (nonce/sequence)
- solo bridge autorizado puede mintear representaciones

27. **MultichainRouter.sol (MAY)**
- enruta transfers/bridges, mantiene mapping de chainIds y bridges.

---

5) Interfaces internas versionadas (API interna)

Objetivo: módulos intercambiables sin tocar el token. Versiona: `V1` (y futuro `V2`).

5.1 Interfaces MUST

- **IAgriComplianceV1**
  - `canTransact(address) external view returns (bool)`

    - `canTransfer(address from, address to, uint256 amount) external view returns (bool)`
    - (ext) `transferStatus(...) external view returns (bool ok, uint8 code)` (Reason Codes)
- **IAgriDisasterV1**
    - `campaignFlags(bytes32 campaignId) external view returns (uint256 flags)`
    - `isRestricted(bytes32 campaignId) external view returns (bool)`
- **IAgriOracleBaseV1**
    - `latestEpoch(bytes32 campaignId) external view returns (uint64)`
    - `isReportValid(bytes32 campaignId, uint64 epoch) external view returns (bool)`
- **IAgriCustodyV1**
    - `isCustodyFresh(bytes32 campaignId) external view returns (bool)`
- **IAgriFundingV1 / ISettlementQueueV1**
    - requests + process + cancel + status
- **IAgriTraceV1**
    - `emitTraceEvent(...)` / `anchorRoot(...)`
- **IAgriBridgeV1** (si multichain)
    - `bridgeOut(...)`, `bridgeIn(...)`, `nonceState(...)`

5.2 Errores y eventos estándar (kit)

- Custom errors con prefijo por módulo (`Agri__...`)
- Eventos normativos:
    - `Frozen(account, amount)`
    - `ForcedTransfer(from,to,amount)`
    - `DisasterDeclared/Confirmed/Escalated/Cleared`
    - `RequestCreated/Processed/Cancelled`
    - `OracleReportSubmitted`
    - `DocRegistered`
    - `TraceEvent` / `BatchRootAnchored`
    - `BridgeOut/BridgeIn`

---

6) Checklist normativo (RFC style)

MUST

- ERC-7943 completo + ERC165 interfaceId correcto.
- Views no revert, fail-closed con safe-staticcall.
- Separación de roles: compliance ≠ disaster ≠ treasury ≠ regulator.
- Request/Process funding+redemption.
- Proceeds-based distribution (cashflow respaldado).
- Disaster levels + TTL + dual-control para hard actions.
- Trazabilidad: hashes+eventos + DocumentRegistry (al menos).

SHOULD

- Factory+Clones default.
- Custody proofs gating en perfiles institucionales/delivery.
- Merkle anchoring para alto volumen.
- Timelock para policy changes.
- InsurancePool extension oficial.

MAY

- Instant mode
- Chainlink adapters
- ZK proofs
- Auctions/RFQ marketplace
- ERC-4626 wrapper para composabilidad financiera

---

7) Conformance Suite (lo que lo “certifica”)

Estructura recomendada (Foundry):

7.1 Conformance ERC-7943

- `test/conformance/ERC7943Fungible.t.sol`
  - no-revert views
  - no storage writes
  - freeze correctness
  - forcedTransfer judicial fuerte (unfreeze-before-transfer si aplica)
  - supportsInterface

7.2 Conformance uAgri (agricultura)

- `test/conformance/AgriCampaignLifecycle.t.sol`
- `test/conformance/RequestProcessFunding.t.sol`
- `test/conformance/ProceedsDistribution.t.sol`
- `test/conformance/DisasterPolicy.t.sol`
- `test/conformance/CustodyGating.t.sol` (si perfil)
- `test/conformance/TraceabilityAnchors.t.sol`

7.3 Invariants (property based)

- `test/invariant/NoRevertViews.t.sol`
- `test/invariant/NoPrivilegeEscalation.t.sol`
- `test/invariant/ValueConservationPerEpoch.t.sol`
- `test/invariant/NoDoubleSettlement.t.sol`
- `test/invariant/DisasterTTLAndDualControl.t.sol`
- `test/invariant/BridgeSupplyInvariant.t.sol` (si bridge)

7.4 Badge de conformidad (CI)

- Script `ci/conformance.sh` que falla si:
  - ERC-7943 no pasa
  - uAgri invariants no pasan
  - gas snapshots (opcional) empeoran sobre umbral

---

8) Invariantes agrícolas “con peso” (norma)

(Resumen normativo; se testea como invariants)

- **Proceeds-backed**: no se distribuye más de lo recibido/registrado.
- **No double counting** por epoch/reportHash.
- **Custody freshness gating** (si habilitado): sin custodia vigente no hay redeem/claim.
- **Disaster TTL**: activación rápida expira si no hay confirmación.
- **Dual-control**: hard freeze / compensación global / swaps críticos solo con multisig/timelock.
- **Campaign state machine** cerrada.
- **Bridge supply invariant** (si multichain).
- **Audit trail**: toda acción crítica con `reasonHash/reportHash` anclado.

---

9) Threat model y runbooks operativos (MUST en docs)

9.1 Threat model (mínimo)

- Compromiso de admin / llaves
- Abuso de DISASTER_ADMIN (mitigado por TTL + confirmación)
- Oracle fraud / custodian fraud (mitigado por multi-provider + evidencias + challenge window opcional)
- Reentrancy y external call hazards (CEI, pull-payments, safe transfer)
- Bridge attacks (nonce, canonical authority, replay protection)
- DoS por colas/listas (max pending, batching, gas caps)

9.2 Runbooks

- Rotación de llaves (two-step)
- Activación y confirmación de desastre
- Pausas y reanudación
- Reemplazo de operador
- Procedimiento de compensación (InsurancePool)
- Proceso de upgrade (si UUPS/hybrid)
- Respuesta a incidentes (playbook de emergencia)

---

10) Optimización y seguridad (reglas del kit)

- **CEI siempre** en funciones que muevan fondos/tokens externos.
- **Pull payments** para claims/compensaciones cuando haya muchos usuarios.
- **Safe ERC20 transfers** (manejar tokens no estándar).
- **Cachear lecturas** (SLOADs) y evitar loops no acotados.
- **Packing** en colas/requests tras tener tests verdes.
- **Assembly solo si hotspot real** y con disciplina memory-safe (preferible evitar en V1).

Orden de ingeniería (y el PASO 1)

✅ PASO 1 (lo primero que hacemos)

Congelar oficialmente la API V1 y sus constantes, basándonos en:

- lo que ya fijamos (receta v1A/v1B),

- y los patrones del FundKit (gas stipend + fail-closed helpers).

Entregables del Paso 1 (sin implementar lógica todavía):

1. `docs/standard.md` final (ya casi lo tenemos, solo lo pulimos con tablas normativas).
2. `src/interfaces/v1/` completo (lista abajo).
3. `src/interfaces/v1/UAgriTypes.sol` + `UAgriErrors.sol` + `UAgriRoles.sol` + `UAgriFlags.sol` + `UAgriReasonCodes.sol` + `UAgriHazards.sol`.
4. "Decision locks": qué interfaces son **MUST** vs **OPTIONAL** por perfil.

   Este paso es el "punto de no retorno": con la API congelada, ya podemos implementar contrato por contrato sin romper compatibilidad.

Roadmap después del Paso 1 (pasos 2..N)

2. Core security primitives: `RoleManager`, `EmergencyPause`, `FreezeManager`, `ForcedTransferController` (sin campaign ops aún).
3. `AgriShareToken` (ERC20+ERC7943) con:
   - fail-closed views,
   - gas stipend config,
   - wiring módulos,
   - enforcement transfer/pause/freeze.
4. `AgriCampaignRegistry` + state machine (FUNDING→ACTIVE→HARVESTED→SETTLED→CLOSED).
5. `SettlementQueue` + `FundingManager` + `CampaignTreasury` (request/process end-to-end, CEI).
6. Distribución: `YieldAccumulator` (unicidad epoch/reportHash + funds-backed).
7. Oráculos EIP-712: Harvest / SalesProceeds / Custody (+ scripts de firma).
8. `DisasterModule` TTL + confirmación dual-control + flags normativos.
9. Trace: `TraceabilityRegistry` + `BatchMerkleAnchor` + `DocumentRegistry`.
10. Factory+Clones + perfiles de despliegue (A/B/C/D).
11. Extensiones: Marketplace / DeliveryReceipts / InsurancePool / BridgeAdapterCanonical.
12. Tests: conformance + invariants + mocks (reutilizando estructura FundKit).

---

3) Interfaces V1 definitivas (APIs con las que trabajaremos)

3.1 Core MUST (V1)

1. **`IERC7943Fungible`**
    - canTransact, canTransfer, getFrozenTokens (no revert)
    - setFrozenTokens, forcedTransfer + eventos/errores
2. **`IAgriModulesV1`** *(añadida tras ver FundKit)*
    - `modules()` / getters (compliance, disaster, freeze, custody, etc.)
    - `viewGasLimits()` (compliance/custody/etc.)
    - `setViewGasLimits(...)` admin (perfil institucional/hybrid)
3. **`IAgriComplianceV1`**
    - canTransact, canTransfer
    - `transferStatus` (Reason Codes) **SHOULD** (puede ir como sub-interfaz `IAgriReasonCodesV1`)
4. **`IAgriDisasterV1`**
    - `campaignFlags(campaignId)`
    - `isRestricted/isHardFrozen`
5. **`IAgriFreezeV1`**
    - `getFrozenTokens(account)` (view, no revert)
6. **`IAgriCampaignRegistryV1`** *(faltaba en tu lista; la añado)*
    - `getCampaign(campaignId)`
    - `state(campaignId)`
    - parámetros core (settlementAsset, windows, plotRef, etc.)
7. **`ISettlementQueueV1`**
    - requestDeposit/requestRedeem/cancel/getRequest/batchProcess
8. **`IAgriTreasuryV1`** *(añadida)*
    - recepción y pagos controlados
    - “purpose tags” + gating hooks (restricted/custody)
9. **`IAgriDistributionV1`** *(añadida)*
    - `notifyReward(amount, epoch, reportHash)`
    - `claim()` y/o `claimFor(account)` (si permitimos relayers)
    - `claimed(account)`/`rewardDebt` (según modo)
10. **`IAgriOracleBaseV1`** + específicos:

- Harvest, SalesProceeds, Custody, DisasterEvidence (cada uno extiende base o comparte base)

11. **`IAgriCustodyV1`**

- `isCustodyFresh(campaignId)`
- `lastCustodyEpoch(campaignId)`

12. **`IAgriTraceV1`**

- emitTrace + anchorRoot + eventos normativos

13. **`IAgriDocumentRegistryV1`** *(añadida)*

- `registerDoc(docType, docHash, refs...)` + evento

3.2 Optional modules (V1)

14. `IAgriBridgeV1` (si multichain activado)
15. `IAgriMarketplaceV1` (reference marketplace)
16. `IAgriDeliveryV1` (delivery receipts / redeem físico)
17. `IAgriInsuranceV1` (payouts por desastre)

Por qué añado estas nuevas interfaces (modules/registry/treasury/distribution/docs): Porque FundKit demuestra que el “estándar de facto” necesita **APIs internas claras** para sustituir módulos sin reescribir el token ni los tests.

---

4) Árbol completo de contratos (repo target final)

Aquí va el árbol “definitivo” (lo puedes usar como checklist). Es prácticamente tu receta v1A, pero con los añadidos de interfaces/constantes y carpetas shared/tests como FundKit:

```
uAgri-ERC7943-StandardKit/
  README.md
  LICENSE
  foundry.toml

  docs/
    standard.md
    profiles.md
    threat-model.md
    runbooks/
      emergency.md
      disaster.md
      key-rotation.md
      upgrades.md
      bridge.md
    architecture/
      flows.md
      diagrams.md

  src/
    interfaces/
      v1/
        IERC7943Fungible.sol
        IAgriModulesV1.sol
        IAgriComplianceV1.sol
        IAgriDisasterV1.sol
```

```
    IAgriFreezeV1.sol
    IAgriCampaignRegistryV1.sol
    ISettlementQueueV1.sol
    IAgriTreasuryV1.sol
    IAgriDistributionV1.sol
    IAgriOracleBaseV1.sol
    IAgriCustodyV1.sol
    IAgriTraceV1.sol
    IAgriDocumentRegistryV1.sol
    IAgriBridgeV1.sol          (optional)
    IAgriMarketplaceV1.sol     (optional)
    IAgriDeliveryV1.sol        (optional)
    IAgriInsuranceV1.sol       (optional)

  constants/
    UAgriTypes.sol
    UAgriErrors.sol
    UAgriRoles.sol
    UAgriFlags.sol
    UAgriReasonCodes.sol
    UAgriHazards.sol

_shared/
  SafeERC20.sol
  ECDSA.sol
  EIP712.sol
  ReentrancyGuard.sol
  SafeStaticCall.sol      (helper tipo FundKit: staticcall+gas stipend+strict returns)
  PackedUint.sol          (si luego hacemos packing serio)
  TwoStepAdmin.sol        (si usamos admin fallback)

access/
  RoleManager.sol

control/
  EmergencyPause.sol
  FreezeManager.sol
  ForcedTransferController.sol

core/
  AgriShareToken.sol

campaign/
  AgriCampaignRegistry.sol
```

```
  FundingManager.sol
  SettlementQueue.sol
  CampaignTreasury.sol

distribution/
  YieldAccumulator.sol
  SnapshotModule.sol

compliance/
  ComplianceModuleV1.sol
  IdentityAttestation.sol
  profiles/
    EUProfileEUDRCompliance.sol

disaster/
  DisasterModule.sol
  InsurancePool.sol            (optional-but-official)

trace/
  TraceabilityRegistry.sol
  BatchMerkleAnchor.sol
  DocumentRegistry.sol

oracles/
  base/
    OracleBaseEIP712.sol
  HarvestOracle.sol
  SalesProceedsOracle.sol
  CustodyOracle.sol
  DisasterEvidenceOracle.sol
  adapters/
    ChainlinkAdapter.sol       (optional)

factory/
  CampaignFactory.sol
  CampaignTemplate.sol
  ClonesLib.sol                (si vamos a EIP-1167)

bridge/
  BridgeAdapterCanonical.sol
  MultichainRouter.sol         (optional)

extensions/
  marketplace/
```

```
      ReferenceMarketplace.sol
    delivery/
      DeliveryReceipt1155.sol
      PhysicalDeliveryModule.sol

test/
  conformance/
    ERC7943Fungible.t.sol
    UAgriStandard.t.sol
    DisasterPolicy.t.sol
    CompliancePolicy.t.sol
    TraceabilityAnchors.t.sol
    CustodyGating.t.sol
    BridgeSupplyInvariant.t.sol  (if enabled)

  invariant/
    NoRevertViews.t.sol
    NoPrivilegeEscalation.t.sol
    ValueConservationPerEpoch.t.sol
    NoDoubleSettlement.t.sol
    DisasterTTLAndDualControl.t.sol
    BridgeReplayProtection.t.sol

  mocks/
    MockComplianceRevert.sol
    MockComplianceGasBomb.sol
    MockFreezeRevert.sol
    MockFreezeGasBomb.sol
    MockERC20.sol

script/
  DeployProfileA.s.sol
  DeployProfileDFactory.s.sol
  SignHarvest.s.sol
  SignSales.s.sol
  SignCustody.s.sol

ci/
  conformance.sh
  gas-snapshot.sh
```

Este es el docs/standard.md

uAgri ERC-7943 Agricultural Standard (uAgri-7943) — v1B

Status: Working De-Facto Standard (Open-Source Kit + Conformance Suite)
Solidity: 0.8.33 (pinned)

1. Scope
uAgri-7943 standardizes:
- Fungible campaign shares (ERC-20 + ERC-7943 enforcement)
- Funding + Redemption (Request/Process MUST; Instant SHOULD)
- Proceeds-based distribution (cashflow-backed)
- Disaster policy (TTL + dual-control + severity levels)
- Compliance (multi-jurisdiction, pluggable, hybrid KYC gating)
- Traceability (hash/event anchoring + Merkle roots SHOULD)
- Multichain bridging (optional module; supply invariants)

Non-goals (CORE):
- On-chain PII storage
- Mandating one marketplace implementation
- Mandating real-world legal structures

2. Definitions
Campaign: an economic unit for a season/lot ("1 campaign = 1 ERC-7943 token").
Settlement Asset: canonical payout asset per campaign (SHOULD be a stable ERC-20).
Restricted Mode: campaign-level operational lock.
Hard Freeze: campaign-level transfer halt.
Fail-closed: treat unknown/missing module responses as NOT allowed.

3. ERC-7943 Non-Negotiables (MUST)
Token MUST implement:
- ERC-20
- IERC7943Fungible
- ERC-165 supportsInterface

Views MUST:
- canTransact(account), canTransfer(from,to,amount), getFrozenTokens(account)
  - MUST NOT revert
  - MUST NOT write storage
  - MUST fail-closed when dependent modules revert / out-of-gas / malformed return

Enforcement MUST:
- transfer/transferFrom MUST revert if canTransfer returns false.
- When transfers are paused, canTransact/canTransfer MUST return false.

Freeze semantics MUST:
- canTransfer MUST include: amount <= balance(from) - frozen(from)
- getFrozenTokens MAY exceed balance

Forced transfer policy (Judicial Strong) MUST:
- forcedTransfer MAY bypass canTransfer
- If it affects frozen tokens, it MUST unfreeze before transferring and emit Frozen before ForcedTransfer.

4. Roles & Separation of Duties (MUST)
Minimum roles:
- DEFAULT_ADMIN (ideally Timelock)
- GUARDIAN (pauses)
- TREASURY_ADMIN
- COMPLIANCE_OFFICER
- DISASTER_ADMIN
- REGULATOR_ENFORCER (forcedTransfer, mass freeze)
- ORACLE_UPDATER
- CUSTODY_ATTESTER
- FARM_OPERATOR
- UPGRADER (if upgradeable)
- BRIDGE_OPERATOR (if bridging enabled)

Critical actions SHOULD require dual-control:
- Mass freeze, hard freeze escalation, forced transfer batches, module swaps, insurance global payouts.

5. Campaign Lifecycle (MUST)
States: FUNDING -> ACTIVE -> HARVESTED -> SETTLED -> CLOSED
Rules:
- Deposits processed only in FUNDING (unless profile overrides)
- Redemptions processed only when allowed by policy (restricted mode gating)
- Claims processed only when allowed AND proceeds are notified

6. Funding/Redemption (MUST)
Request/Process MUST be supported:
- requestDeposit, requestRedeem, cancel, batchProcess
Anti-spam MUST:
- maxPendingPerAccount
- deadlines/expirations
- no double-processing

Instant SHOULD be supported as a profile:

- only if price model is safe (fixed price / robust oracle / bounded slippage)

7. Proceeds-Based Distribution (MUST)
Distributions MUST be cashflow-backed:
- notifyReward(amount, epoch, reportHash) MUST be backed by funds received by treasury (or verifiable escrow in institutional profile)
No double counting MUST:
- (campaignId, epoch, reportHash) uniqueness

Value conservation MUST:
- sum(claimsPaid) <= proceedsNotified - fees/reserves for the same epoch

8. Disaster Standard (MUST)
Hazard Codes (base set):
- DROUGHT, FLOOD, HAIL, FIRE, PEST_DISEASE, FROST, STORM, EARTHQUAKE
- WATER_SUPPLY_FAILURE, GOV_ACTION, EXPROPRIATION_OR_ACCESS_LOSS
- SUPPLY_CHAIN_DISRUPTION, FRAUD_OR_MATERIAL_BREACH

Severity: 0..4

Activation policy MUST:
- DISASTER_ADMIN can declare with TTL (temporary)
- Extended/Hard actions MUST be confirmed by governance (multisig/timelock)

Levels:
- ALERT: soft restrictions
- RESTRICTED: default pauses redemption/claims
- HARD_FREEZE: pauses transfers (only judicial/maintenance allowed)

TTL MUST:
- temporary declarations expire automatically if not confirmed.

9. Compliance Standard (MUST)
Compliance module MUST be pluggable and support:
- sanctions deny
- jurisdiction allow/deny
- investor tier + caps
- holding cap
- lockupUntil
- optional: daily transfer cap, transfer windows, corridors, venues allowlist

Hybrid gating SHOULD:
- stricter checks for mint/redeem/claim and large tiers

Identity onboarding SHOULD:
- EIP-712 attestations with providerId, expiry, nonce

EU Profile (EUDR) SHOULD:
- implemented as a regional module

10. Traceability Standard (MUST/SHOULD)
MUST:
- hash/event anchoring for trace events and documents
SHOULD:
- Merkle root anchoring for high-volume telemetry/logistics
MAY:
- ZK proofs
- NFT/1155 receipts for delivery/custody

Privacy:
- PII MUST remain off-chain; on-chain stores hashes and minimal metadata.

11. Oracles (MUST)
MUST oracles:
- Harvest report
- Sales/proceeds evidence
- Custody/inventory proof
- Disaster evidence (optional but recommended)

All oracles MUST:
- use EIP-712 attestations (base)
- enforce anti-replay, monotonic epoch, freshness window, reportHash

Adapters MAY:
- Chainlink/feeds via adapter contracts

12. Reason Codes (SHOULD)
Extensions SHOULD provide:
- transferStatus(from,to,amount) -> (bool ok, uint8 code)

Codes (baseline):
- 0 OK
- 1 PAUSED
- 2 SENDER_RESTRICTED
- 3 RECIPIENT_RESTRICTED
- 4 INSUFFICIENT_UNFROZEN
- 5 COMPLIANCE_DENY
- 6 DISASTER_RESTRICTED

- 7 LOCKUP_ACTIVE
- 8 CAP_EXCEEDED
- 9 ORACLE_STALE
- 10 CUSTODY_STALE
- 255 UNKNOWN_FAIL_CLOSED

13. Multichain Bridge (Optional Module, but KIT MUST include)
Canonical-per-campaign:
- One chain is canonical for supply
- Satellite representations must be fully backed

MUST invariants:
- no free mint
- replay-protected messages
- only authorized bridge can mint/burn representations

14. Conformance Suite (MUST)
Repo MUST include tests:
- ERC-7943 conformance
- uAgri lifecycle, funding queue, proceeds distribution
- disaster TTL + dual-control
- compliance fail-closed views
- trace anchoring
- bridge supply invariants (if enabled)
- invariants: no privilege escalation, value conservation, no double settlement, no-revert views